

09 — Programmverzweigungen

Entscheidungen im Code mit if, elif, else, match

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Verzweigungen?
2. `if` – die Wenn-Anweisung
3. `if-else` – entweder / oder
4. `elif` – mehrere Stufen
5. Bedingungen kombinieren mit `and/or`
6. `match-case` – elegante Fallunterscheidung
7. Anti-Pattern: `== True`, `if x = 5`

Lernziel: Programme treffen Entscheidungen anhand von Vergleichen aus Notebook 08.

Wie wir heute arbeiten: Nach jedem Konzept zeigt die Folie *Live-Demo* (ich im Terminal) und *Sofort ausprobieren* (Sie im Notebook 09).

Wozu Verzweigungen?

Ohne Verzweigung - Programm ist starr

```
print("Versand: 4.95 EUR")    # immer gleich
```

Mit Verzweigung - Programm reagiert

```
if bestellsumme >= 50:  
    print("versandfrei")  
else:  
    print("Versand: 4.95 EUR")
```

Verzweigungen lassen Programme **auf Daten reagieren** - sie sind das, was Code von einer Tabelle unterscheidet.

if - die Wenn-Anweisung

```
bestand = 3  
if bestand < 5:  
    print("Achtung -- Bestand niedrig!")
```

Bestandteile

- if - Schlüsselwort
- bestand < 5 - **Bedingung**, liefert True/False (NB 08)
- : - Doppelpunkt schließt die Bedingung ab
- **Einrückung** (4 Leerzeichen) - markiert was zur if-Anweisung gehört

Bedingung **wahr** → Block läuft. Bedingung **falsch** → Block wird übersprungen.

► **Live-Demo** - 01_if_einfach.py

📎 **Sofort ausprobieren** - Notebook 09, Kap. 2: Volljährigkeit, Notentest, Passwort-Prüfung

Einrückung ist Pflicht

```
if bestellsumme >= 50:
    print("versandfrei")           # gehört zum if
    print("Geschenk dazu")        # gehört zum if
print("Vielen Dank!")            # läuft IMMER
```

- Python erkennt Blöcke an der **Einrückung**
- Konvention: **4 Leerzeichen**, keine Tabs
- `IndentationError`, wenn die Einrückung nicht passt

Andere Sprachen nutzen { } – Python nutzt **Leerzeichen**. Saubere Optik = lauffähiger Code.

if-else - beide Pfade

```
if bestellsumme >= 50:  
    print("versandfrei")  
else:  
    print("Versand: 4.95 EUR")
```

- **Genau einer** der beiden Blöcke läuft
- else braucht keine eigene Bedingung – “alles übrige”
- Auch hier: Doppelpunkt + Einrückung

► **Live-Demo** – 02_if_else_versand.py

✎ **Sofort ausprobieren** – Notebook 09, Kap. 3: gerade/ungerade, positiv/negativ, Volljährigkeit

elif - mehrere Stufen

```
if bestellsumme >= 100:  
    rabatt = 0.10  
elif bestellsumme >= 50:  
    rabatt = 0.05  
else:  
    rabatt = 0.0
```

- Python prüft die elif-Bedingungen **der Reihe nach** von oben nach unten
- Sobald eine wahr ist, läuft ihr Block – der Rest wird **übersprungen**
- elif so viele wie nötig

Wichtig: Bedingungen **disjunkt** halten – die spezifischste zuerst, die allgemeinste zuletzt.

► **Live-Demo** - 03_elif_rabattstufen.py

Bedingungen kombinieren - and, or, not

```
if stammkunde and bestellsumme >= 50:  
    print("VIP-Versand")  
elif bestellsumme >= 50 or premium:  
    print("Standard-Versand kostenlos")  
else:  
    print("Versand 4.95 EUR")
```

- Operatoren aus Notebook 08 leben hier auf
- Klammern erhöhen Lesbarkeit – besonders bei and/or-Mix

► **Live-Demo** – 04_kombinierte_bedingungen.py

🔗 **Sofort ausprobieren** – Notebook 09, Kap. 4 (mehrere): Notenstufen, Wettervorhersage, Passwort-Stärke

match-case - elegante Fallunterscheidung

```
modell = "Eco-Sneaker"
```

```
match modell:  
    case "Eco-Sneaker":  
        print("Klassiker -- recyceltes Polyester")  
    case "Hemp-High":  
        print("Knöchelhoch -- aus Hanf")  
    case "Bambus-Boot":  
        print("Winterfest -- Bambusfaser")  
    case _:  
        print("Modell nicht im Sortiment")
```

- case `_`: ist der **Default-Fall** (wie `else`)
- Ideal für **gleichwertige, exakte** Vergleiche
- Verfügbar **ab Python 3.10**

► **Live-Demo** - 05_match_case.py

match vs. if-elif

Wann besser match?

viele exakte Werte ("A", "B", ...)
eine Variable, viele Fälle
Code soll übersichtlich bleiben

Wann besser if-elif?

Bereiche ($x < 18$, $x \geq 65$)
mehrere Variablen kombiniert
komplexe Bedingungen mit and/or

besser if-elif -- Bereiche

```
if alter < 18:    rabatt = 0.10
elif alter > 65:  rabatt = 0.15
else:            rabatt = 0.0
```

match vs. if-elif - wann match?

```
# besser match -- exakte Werte  
match plz_region:  
    case "10": ort = "Berlin"  
    case "80": ort = "München"  
    case _:    ort = "unbekannt"
```

Anti-Patterns

```
if storniert == True:           # überflüssig
    ...
if storniert:                   # so reicht's
    ...

if x = 5:                       # SyntaxError -- = ist Zuweisung!
if x == 5:                      # so meinte man's
```

- bool-Werte direkt nutzen, nicht mit True/False vergleichen
- == (Vergleich) und = (Zuweisung) **strikt** trennen
- **Verschachtelte ifs** vermeiden, wenn elif reicht

Code, der **nichts Falsches macht**, ist gut. Code, der zusätzlich **leicht zu lesen** ist, ist sehr gut.

Cheat Card

Aufgabe	Syntax
eine Bedingung	<code>if cond:</code>
zwei Pfade	<code>if cond: ... else: ...</code>
mehrere Stufen	<code>if c1: ... elif c2: ... else: ...</code>
kombinierte Bedingung	<code>if a and b or not c:</code>
exakte Werte	<code>match x: case "a": ... case _: ...</code>
Default-Fall	<code>case _:</code>

Faustregel: zuerst Bedingung in Worten, dann in Python. Wenn Sie's nicht sagen können, können Sie's nicht programmieren.

Ausblick: Notebook 10 – Schleifen

if entscheidet **einmal**. Schleifen **wiederholen**:

```
for produkt in warenkorb:  
    if produkt in lager:  
        print(f"{produkt}: lieferbar")  
    else:  
        print(f"{produkt}: nicht verfügbar")
```

Damit lassen sich ganze Warenkörbe, Bestellungen, Listen prüfen – nicht nur ein Eintrag.

Heute geübt

- ✓ if – bedingt ausführen
- ✓ if-else – entweder / oder
- ✓ elif – mehrstufig prüfen
- ✓ Bedingungen mit and/or/not kombinieren
- ✓ match-case – exakte Werte elegant verzweigen
- ✓ Klassiker-Fallen = vs. == und == True umgangen

Zur Vertiefung im Notebook 09:

- “Sofort ausprobieren“-Aufgaben in Kap. 2-4 + Kap. 6 (sind Sie schon mitgegangen)
- Praktische Übung Kap. 5 / 7: Maschinenauslastung, mehrere Maschinen
- Trainingsmaterial mit gestuften Aufgaben am Notebook-Ende